Conceitos Básicos e System Calls

Professor: Paulo Roberto de Oliveira
Disciplina: Sistemas Operacionais

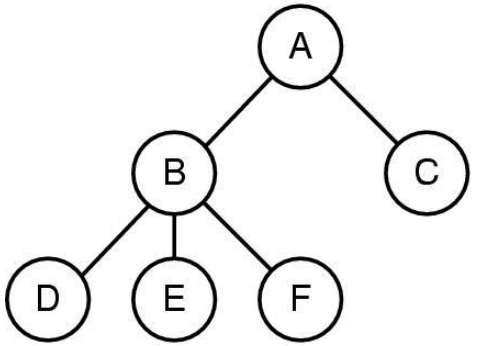
Departamento de Informática
Universidade Estadual de Maringá

Processos

- Processo = Programa em execução.
- Cada processo possui um espaço de endereçamento, uma lista de endereços de memória onde o programa pode ler e gravar dados.
- Cada programa/processo possui um conjunto de registradores e outras informações que devem ser salvas quando o SO decide “entregar” a CPU para outro programa/processo.
- Para isso, cada programa é inserido em uma tabela de processos. Em geral, uma lista ligada com uma entrada para cada processo.

Processos

- Processos são associados com identificadores:
  - Usuários: UID;
  - Processo: PID;
  - Grupo: GID.

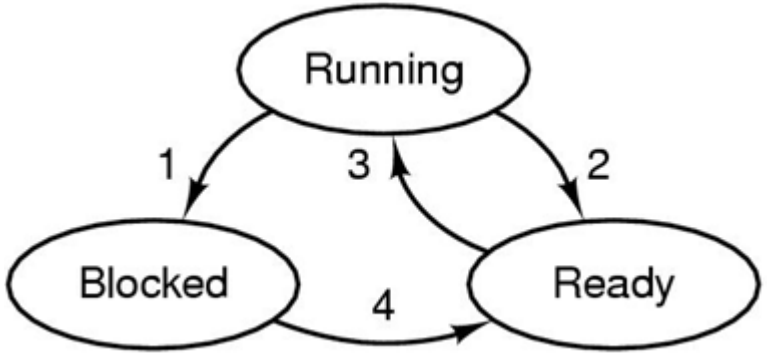


- Processos podem se organizar em hierarquias, desde que um processo possa dar origem a outros.
- O processo que cria outros processos é chamado de processo **pai**, enquanto os criados são chamados de processos **filhos**.

Processos

- Existem 3 estados possíveis para um processo:
  - **Executando**: Usando a CPU neste instante;
  - **Pronto**: Pode executar, mas não há CPU disponível;
  - **Bloqueado**: Incapaz de executar pela espera de algum evento externo.

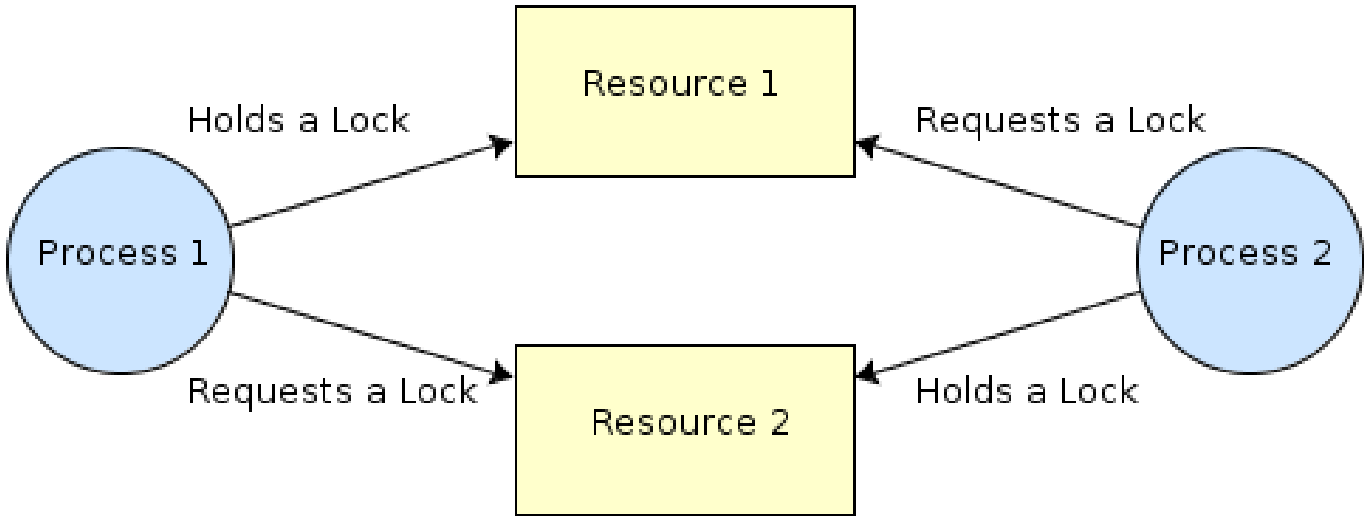
1. O processo é bloqueado para esperar uma entrada de dados.
2. O escalonador escolhe outro processo.
3. O escalonador escolhe este processo.
4. A entrada de dados é realizada.



Deadlocks

- Deadlock é uma situação a que os processos podem chegar por compartilharem recursos de uso exclusivo.
- **Ex.:** Considere 2 processos e 2 recursos, uma impressora e um setor específico do disco rígido;
  - Processo A pede o controle da impressora;
  - Processo B pede o setor do disco;
  - Processo A pede o setor do disco – como está em uso e é exclusivo, o processo é bloqueado;
  - Processo B pede a impressora, e da mesma forma é bloqueado;
  - Como só um dos processos pode fornecer o recurso que o outro precisa e está bloqueado justamente por causa do outro, esse sistema não tem solução – os processos podem ficar nessa situação indefinidamente.

Deadlocks


Resource 1
Holds a Lock
Requests a Lock
Process 1
Process 2
Requests a Lock
Holds a Lock
Resource 2

Gerência de Memória

- Todo computador possui memória principal, usada para manter os programas em execução.
- Para evitar a interferência de um com o outro (e com o SO), há necessidade de algum tipo de proteção.
- Outro ponto importante é o manuseio do espaço de endereçamento de cada processo (faixa de endereçamento).

Gerência de Memória

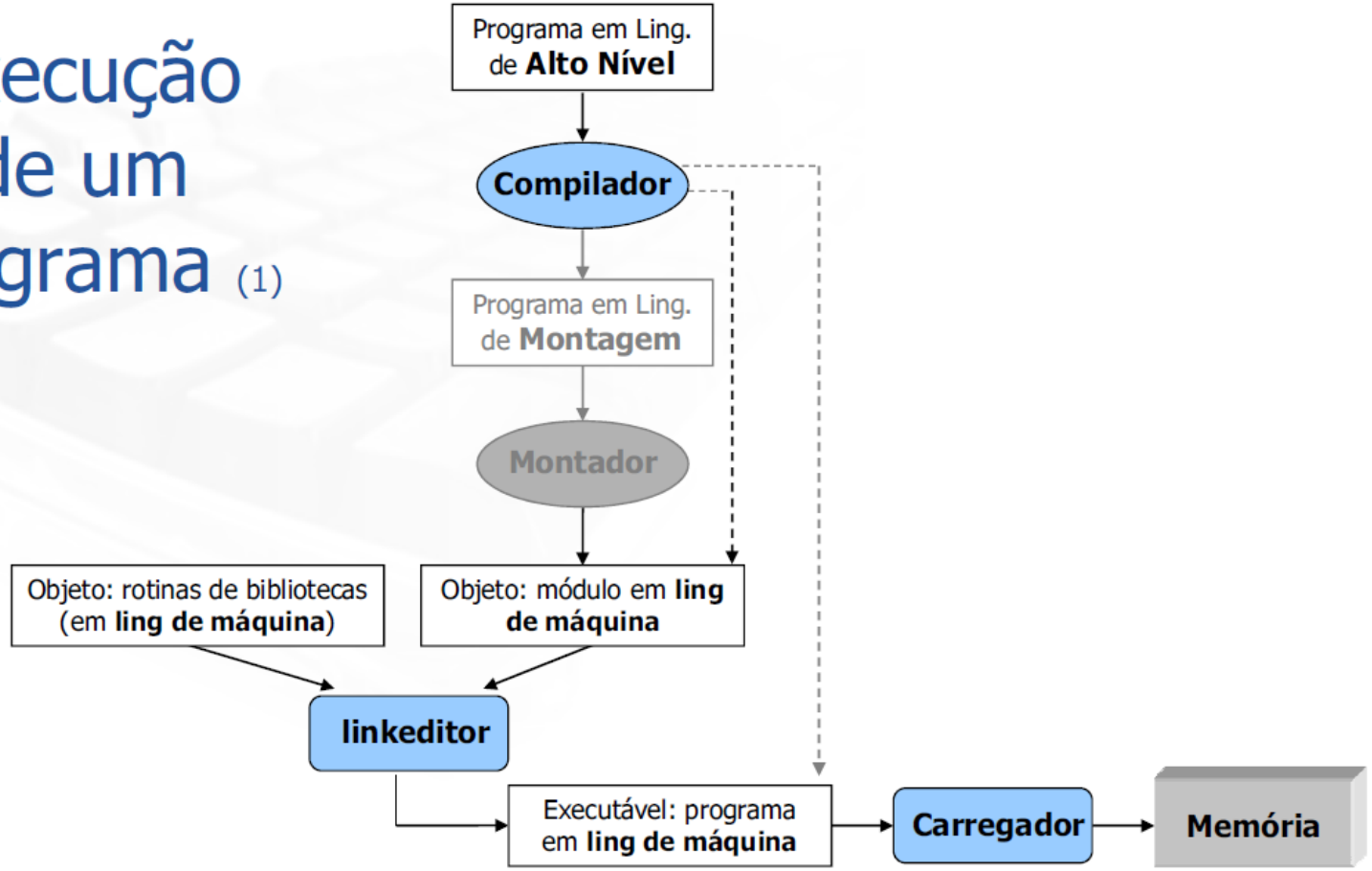
- No caso mais simples, o espaço de um processo deve ser menor que a memória principal;
- Os processadores de 32 bits (e até de 64 bits) permitem espaços de 2^{32} e 2^{64} bytes;
- Se essa memória não existir, a técnica de **memória virtual** utiliza disco para aumentar a capacidade do sistema.

Gerência de Memória

- Algumas funções do Gerenciador de memória do S.O.:
  - Controlar quais as unidades de memória estão ou não estão em uso, para que sejam alocadas quando necessário;
  - Liberar as unidades de memória que foram desocupadas por um processo que finalizou;
  - Tratar do **Swapping** entre memória principal e memória secundária (disco).
  - Transferência temporária de processos residentes na memória principal para memória secundária.

Gerência de Memória

Execução de um Programa (1)



Execução de um Programa (2)

Executável: programa em **ling de máquina**

Espaço de Endereçamento Lógico

Código absoluto:

- Endereços relativos ao início da memória (endereços reais)
- Programas exclusivos para partições específicas na memória

Código relocável

- O programa pode ser carregado em qualquer posição da memória.
- Deve haver uma **tradução** de endereços (ou relocação de endereços)

I/O (Input/Output)

- Há muitos tipos de dispositivos de I/O diferentes;
- Depende do SO o tratamento de cada um;
- Para essa finalidade, existe um subsistema de I/O em cada SO;
- Uma parte é independente de dispositivo;
- O resto é específico de cada um (*device drivers*).

Segurança

- No Unix, os arquivos são protegidos por 9 bits divididos em 3 campos, um para o dono (*owner*), outro para o grupo e outro para todos;
- Cada campo possui 3 bits no formato rwx (leitura-escrita-execução).
- Além da proteção dos arquivos, há muitas outras formas de segurança.
- **Ex.:** Proteção contra invasões, por pessoas e/ou vírus.

Shell

- Um **shell** é um interpretador de comandos do usuário.
- É um programa que não faz parte do SO, mas para o usuário ele é de fato o SO.
- O shell recebe comandos do usuário (ex. executar programas, renomear arquivos, criar diretórios, etc.) e executa-os, caso as permissões estejam OK.
- Ele roda em modo usuário e executa os comandos através de *system calls*.

System Calls

- Interface entre o SO e os programas do usuário.
- Definem o que o SO pode e não pode fazer.
- O conjunto de *system calls* varia de SO para SO, mas algumas são comuns a todos, embora ainda possam variar em nome e parâmetros. **Ex.:** leitura e gravação em arquivos.
- Algumas linguagens são padronizadas e apresentam o mesmo conjunto de *system calls*, independente de SO. **Ex.:** Linguagem C.

System Calls

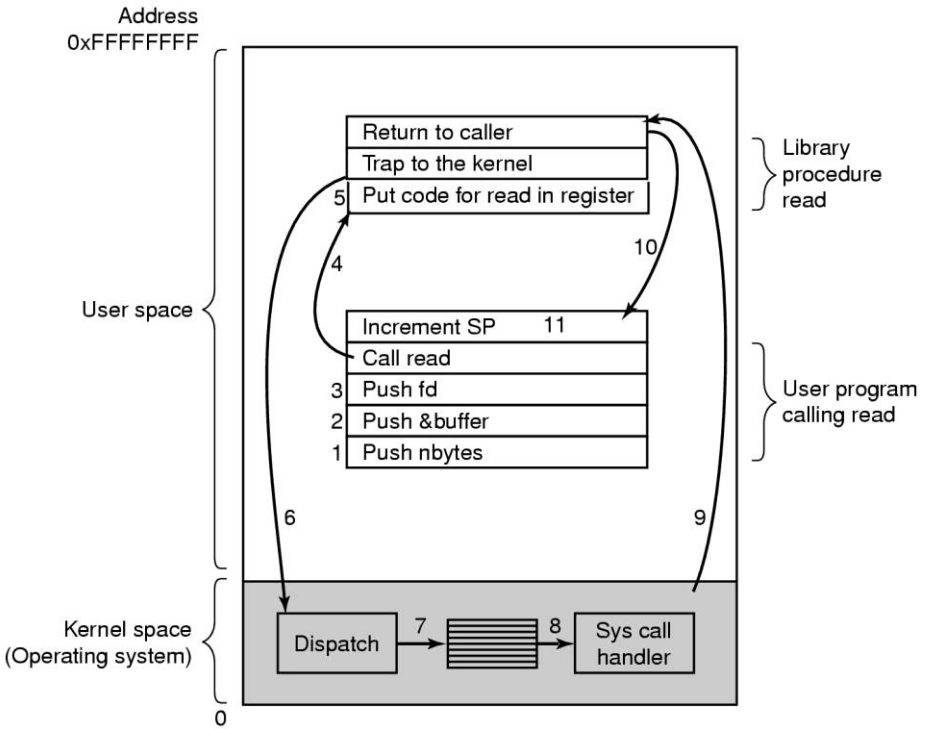
- Se um processo do usuário necessita de um serviço do SO, uma *system call* é usada para solicitar a realização dessa tarefa.
- Porém, isso exige a passagem do processador do modo ***user*** para o modo ***kernel***.
- Isso é realizado na implementação da *system call*, pela colocação de uma instrução chamada ***trap***.
- Um *trap* é uma instrução que causa uma interrupção, mudando o modo do processador e desviando o processamento para uma rotina específica do SO.

System Calls

- A mudança de modo ***user*** para o modo ***kernel*** ocorre em 3 situações:
  - Interrupções: O hardware precisa da intervenção do SO;
  - Exceções: Uma aplicação comete algum erro (divide por 0, etc.);
  - Trap: A aplicação pede um serviço ao SO (system call).
- O retorno ocorre pela execução de uma instrução chamada **RTI**.

System Calls

- Exemplo de código em C: contador = **read**(fd, buffer, nbytes);



1. push nbytes: Coloca na pilha o valor de nbytes;
2. push &buffer: Coloca na pilha o endereço da área onde será feita a cópia dos dados;
3. push fd: Coloca na pilha o descritor do arquivo;
4. call read: Chama a função que faz a leitura (desvio para a biblioteca);
5. Coloca o código da operação num registrador;
6. **<u>trap</u>**: Interrupção por software (A execução é desviada para uma rotina especial do SO, em modo kernel, o **dispatch**);
  6.1. O **dispatch** identifica o código da operação;
7. Numa tabela de ponteiros, o **dispatch** desvia para a rotina que faz o tratamento da operação pedida.
8. A rotina trata o **read**;
9. Retorna para o programa usuário (biblioteca), em modo *user*, para o próximo comando após o trap – o resultado da operação já está disponível para o programa;
10. Retorna para o ponto que chamou o read no programa do usuário;
11. Incrementa a pilha, continua a execução.

Dúvidas?